

CS 251: Software Engineering 1

Lecture 2

Soha Makady

s.makady@fci-cu.edu.eg

Software Engineering: A Working Definition

Software Engineering is a collection of **techniques**, **methodologies** and **tools** that help with the production of

A high quality software system developed with a given *budget* before a given *deadline* while *change* occurs

Challenge: Dealing with complexity and change

Techniques, Methodologies and Tools

- **Techniques:**

- Formal procedures for producing results using some well-defined notation

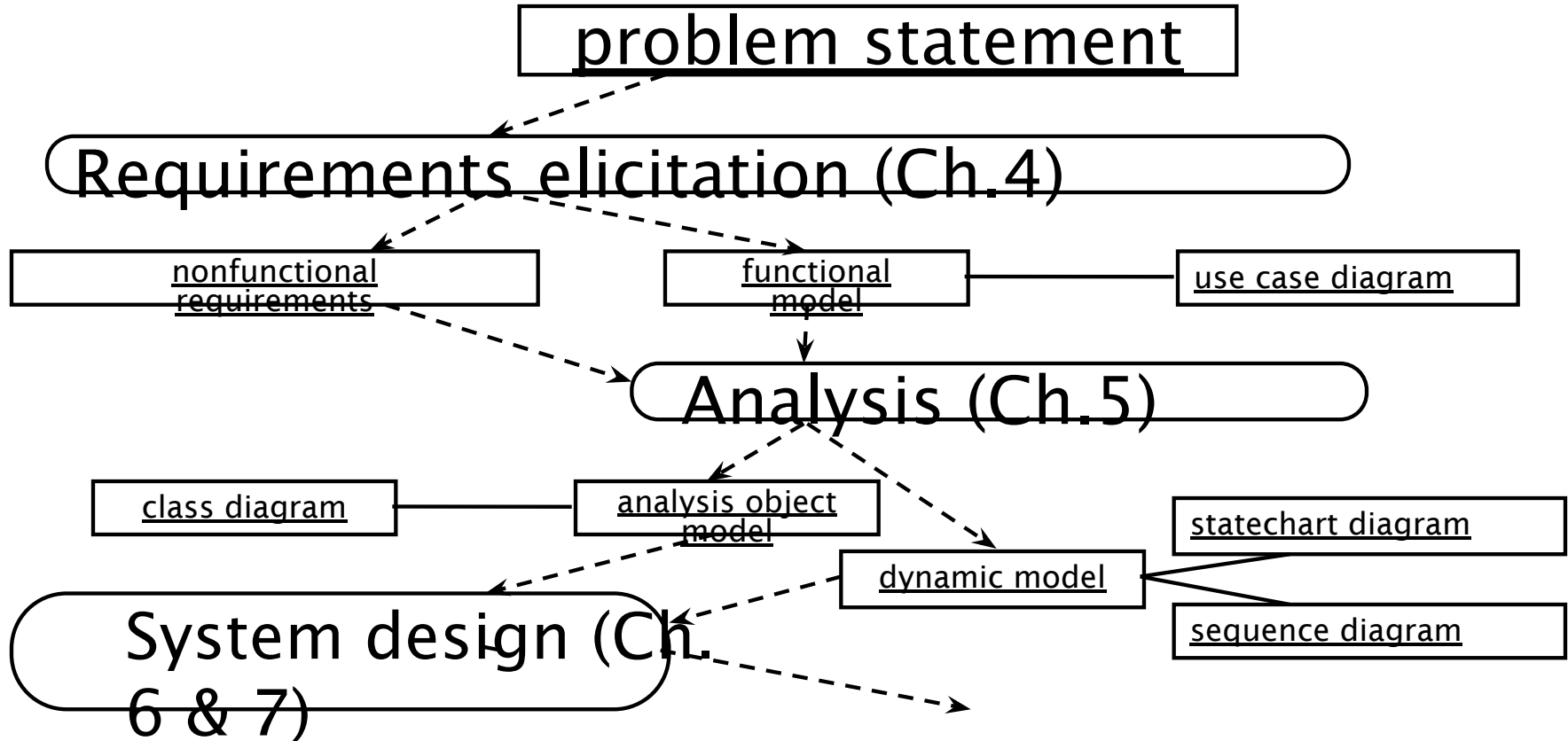
- **Methodologies:**

- Collection of techniques applied across software development and unified by a philosophical approach

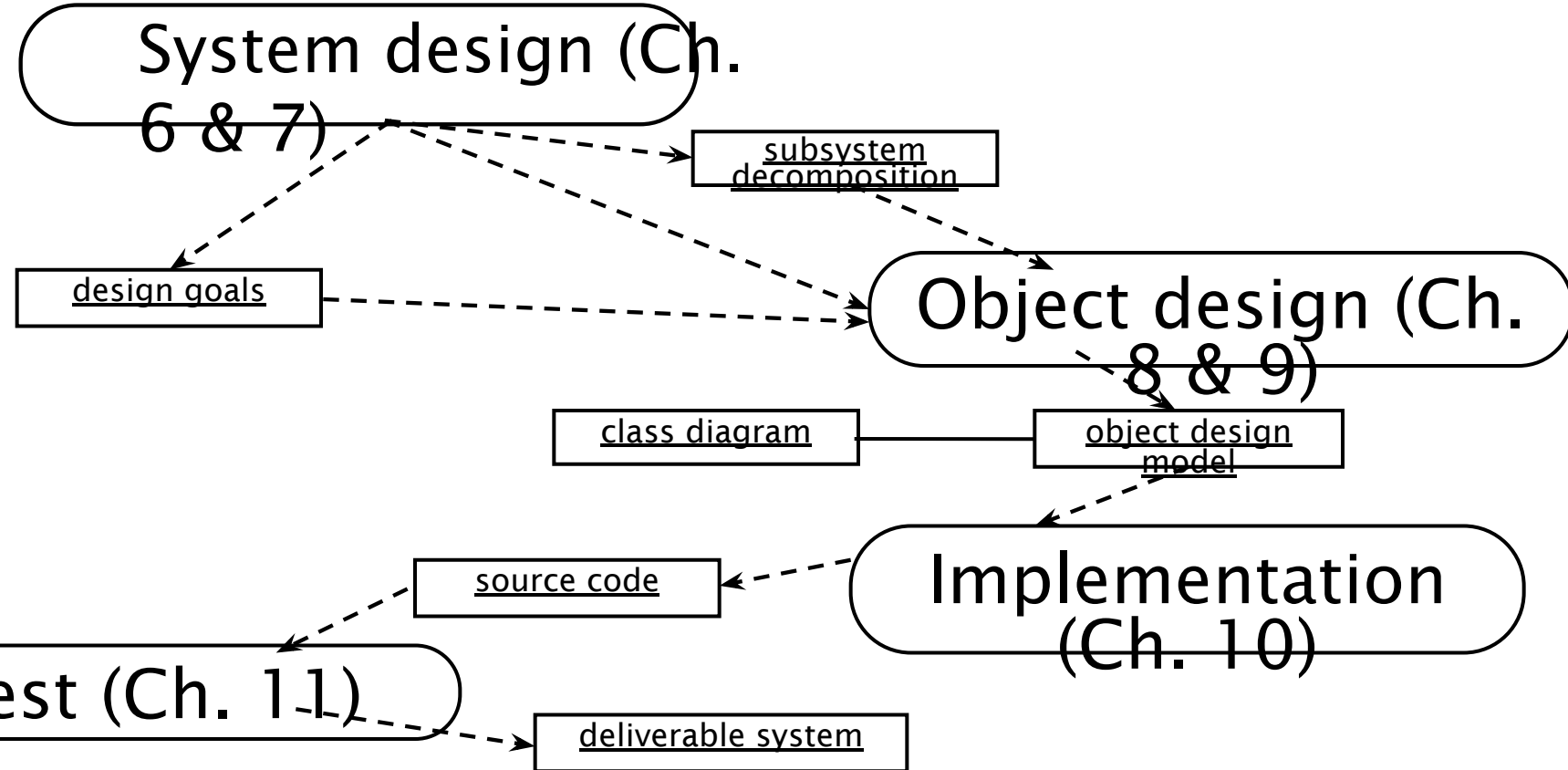
- **Tools:**

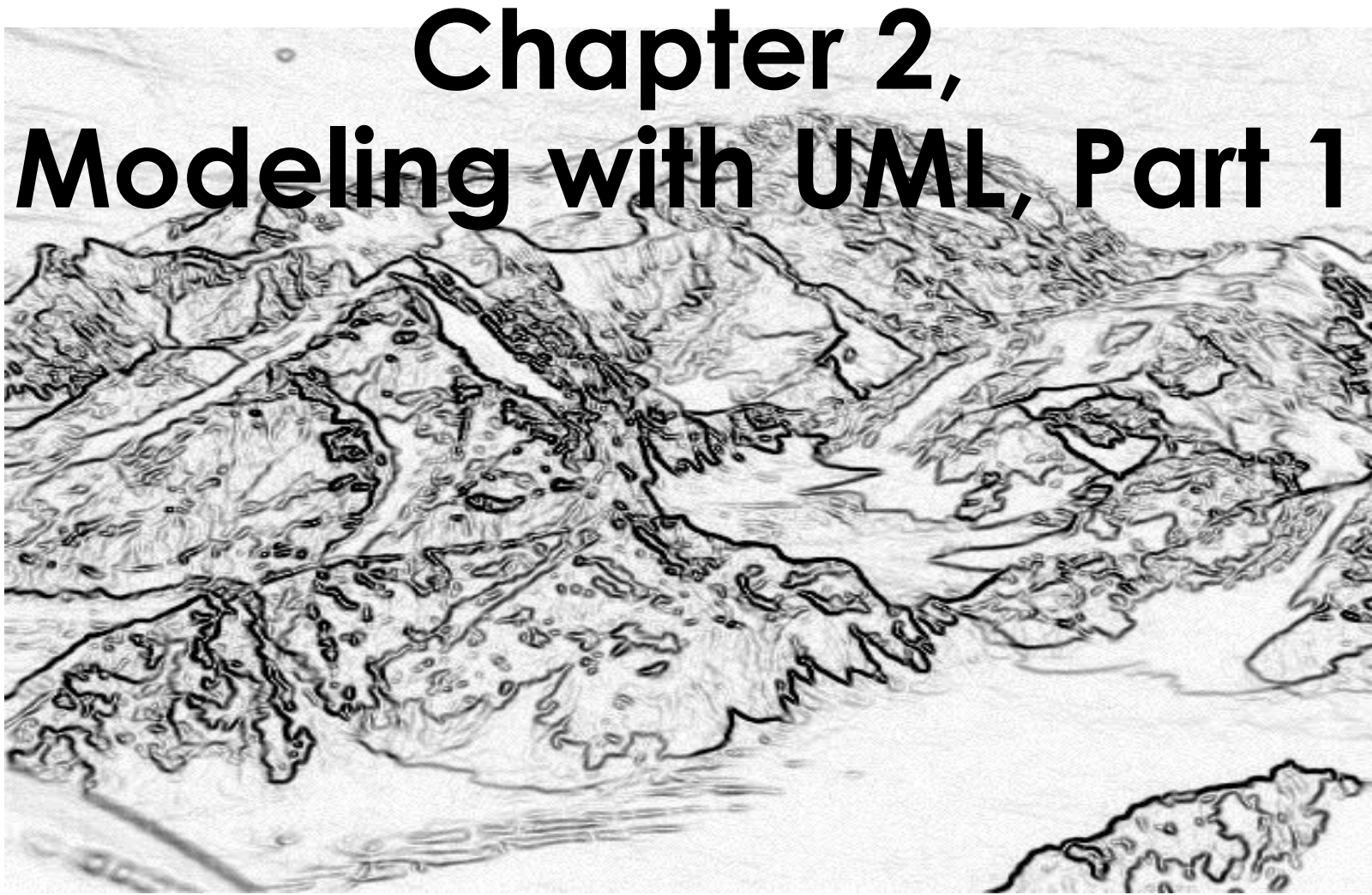
- Instruments or automated systems to accomplish a technique
- CASE = Computer Aided Software Engineering

Software Engineering Development Activities

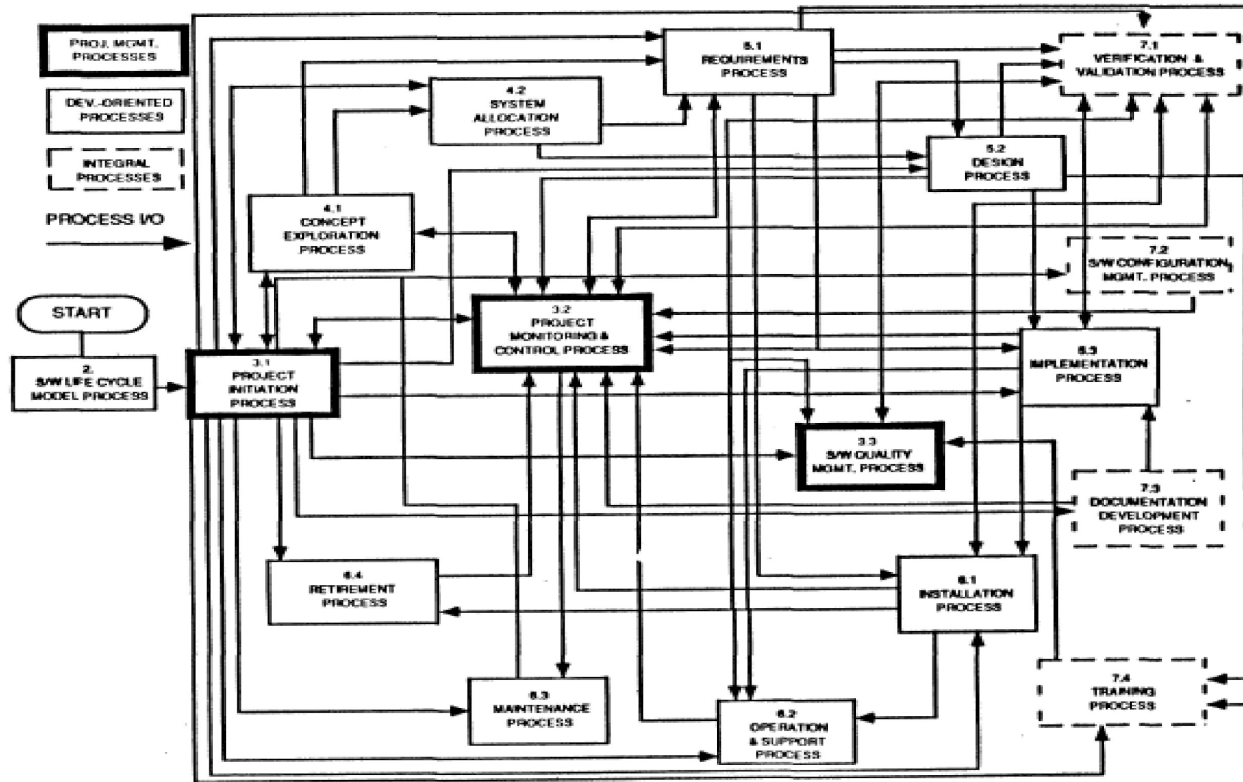


Software Engineering Development Activities





What is the problem with this Drawing?



Overview for the Lecture

- Three ways to deal with complexity
 - **Abstraction and Modeling**
 - Decomposition
 - Hierarchy

Abstraction

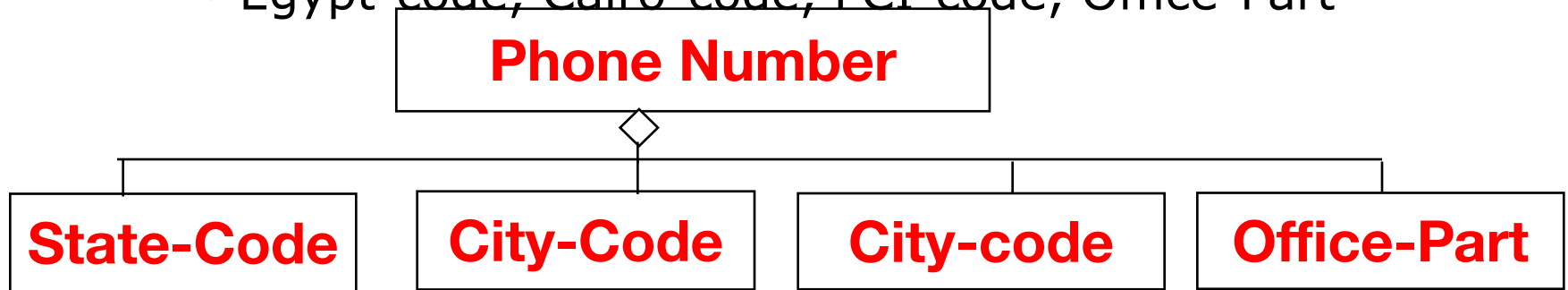
- Complex systems are hard to understand
 - The 7 +- 2 phenomena
 - Our short term memory cannot store more than 7+-2 pieces at the same time -> **limitation of the brain**
 - Phone Number: 498928918204

Abstraction

- Complex systems are hard to understand
 - The 7 +- 2 phenomena
 - Our short term memory cannot store more than 7+-2 pieces at the same time -> limitation of the brain
- Chunking:
 - TUM Phone Number: 498928918204
 - Group collection of objects to reduce complexity
 - 4 chunks:
 - State-code, city-code, TUM-code, Office-Part

Abstraction

- Complex systems are hard to understand
 - The 7 +- 2 phenomena
 - Our short term memory cannot store more than 7+-2 pieces at the same time -> limitation of the brain
- **Chunking:**
 - Phone Number: 2023381687230
 - Group collection of objects to reduce complexity
 - Egypt-code, Cairo-code, FCI-code, Office-Part



What is Modeling?

- ◆ Modeling consists of building an abstraction of reality.
- ◆ Abstractions are simplifications because:
 - ◆ **They ignore irrelevant details and**
 - ◆ **They only represent the relevant details.**
- ◆ What is *relevant* or *irrelevant* depends on the purpose of the model.

Model

- A model is an abstraction of a system
 - A system that no longer exists
 - An existing system
 - A future system to be built.



We use Models to describe Software Systems

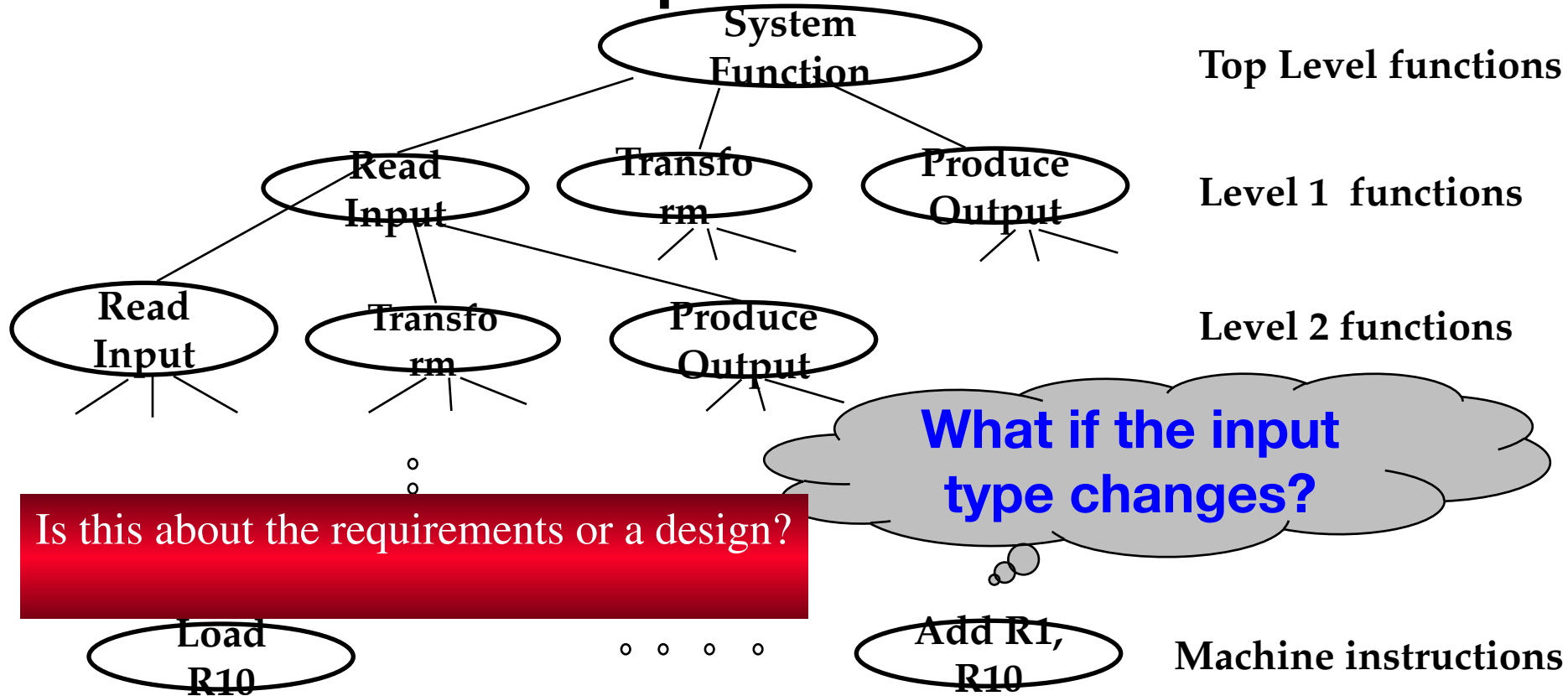
- **Object model:** What is the structure of the system?
- **Functional model:** What are the functions of the system?
- **Dynamic model:** How does the system react to external events?
- **System Model:** Object model + functional model + dynamic model

- Three ways to deal with complexity
 - Abstraction and Modeling
 - **Decomposition**
 - Hierarchy

Decomposition

- A technique used to master complexity (“divide and conquer”)
- Two major types of decomposition
 - Functional decomposition
 - Object-oriented decomposition
- **Functional decomposition**
 - The system is decomposed into modules
 - Each module is a major function in the application domain
 - Modules can be decomposed into smaller modules.

Functional Decomposition



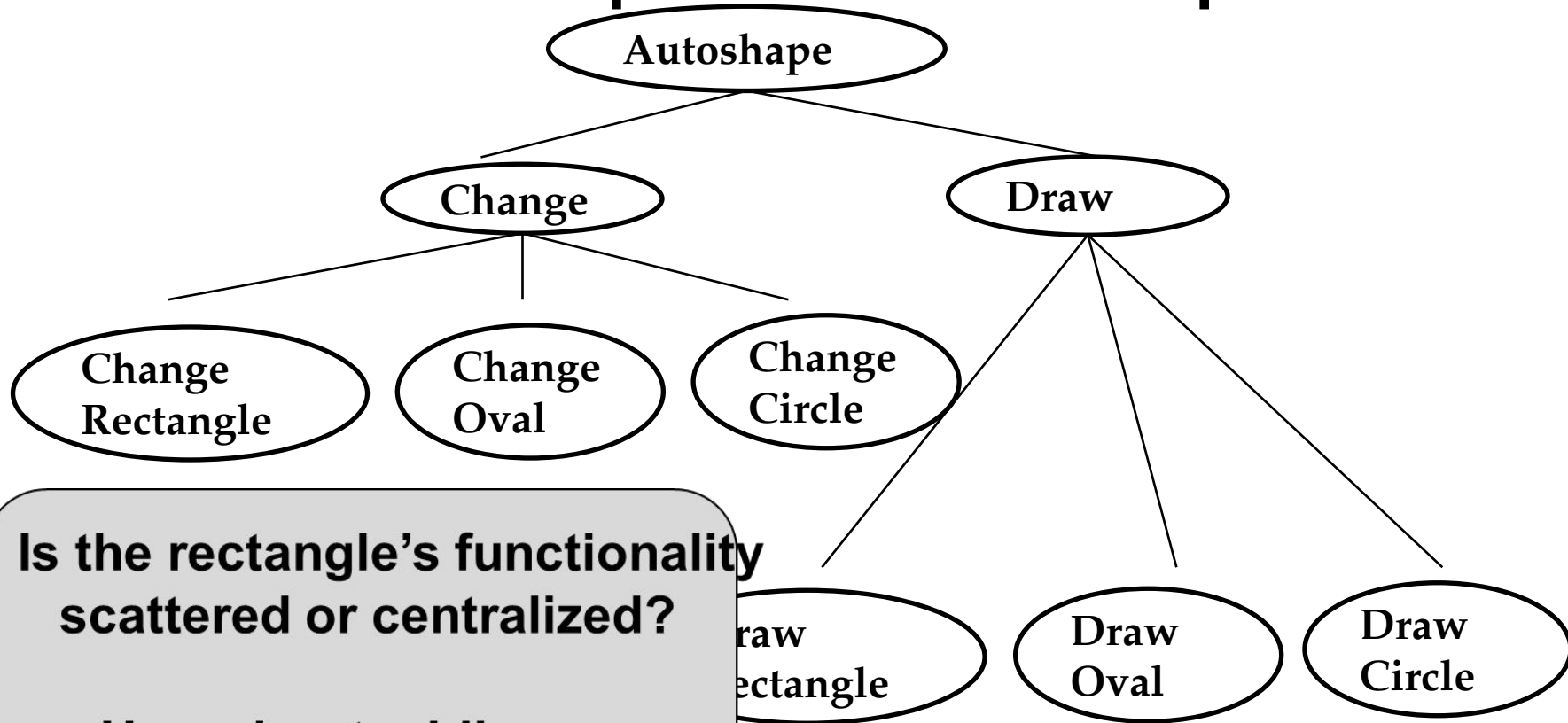
Decomposition (cont'd)

- **Object-oriented decomposition**
 - The system is decomposed into classes ("objects")
 - Each class is a major entity in the application domain
 - Classes can be decomposed into smaller classes
- **Let us see a comparative example**

Functional vs. Object-Oriented Decomposition

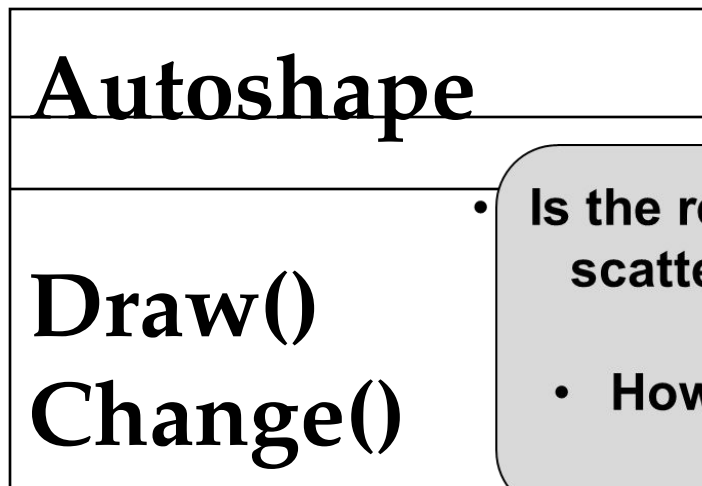
- Example: Consider Microsoft Powerpoint's Autoshapes
- Suppose that you have some shapes that you need to draw
- You also need to change a shape to another shape.
-

Functional Decomposition: Autoshape



- **Is the rectangle's functionality scattered or centralized?**
 - How about adding new shapes?

Object-Oriented View



- Is the rectangle's functionality scattered or centralized?
- How about adding new
 - shapes?

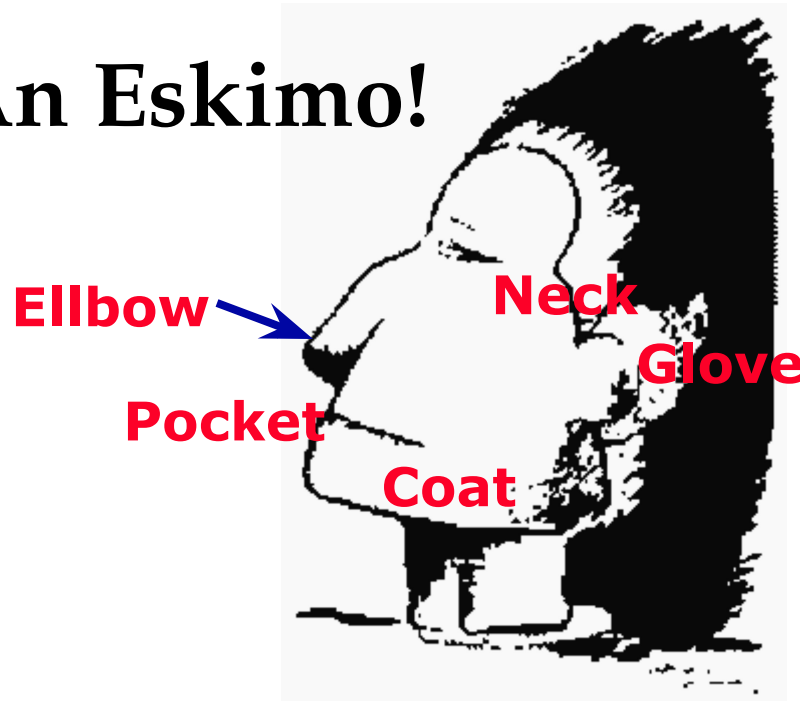
Decomposition (cont'd)

- Object-oriented vs. functional decomposition

Which decomposition is the right one?

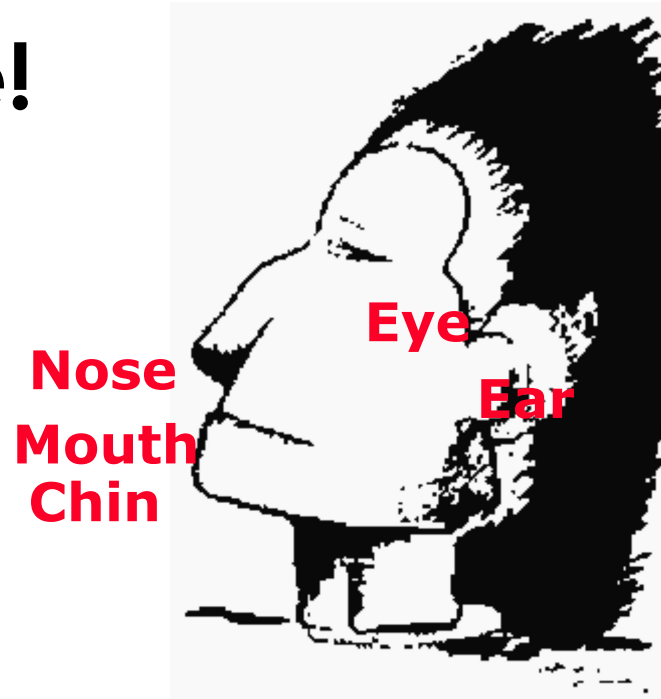
Class Identification - What is This?

An Eskimo!

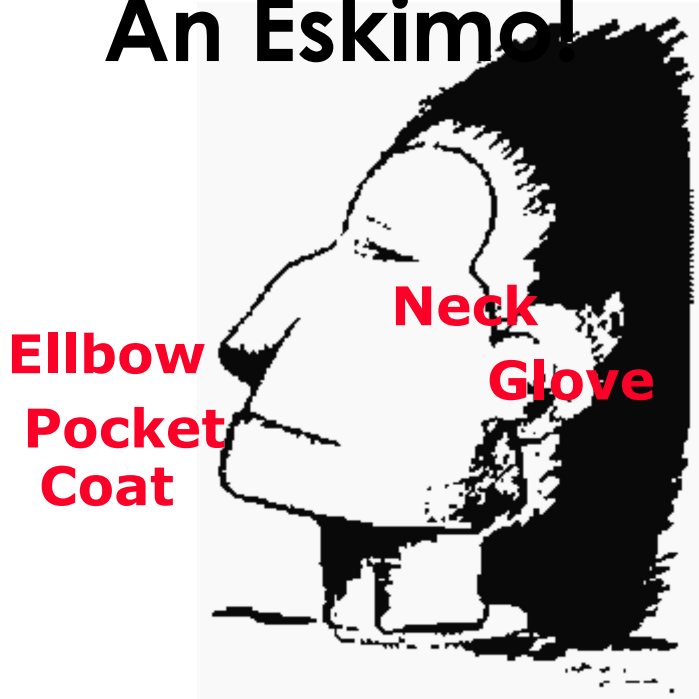


Cave

A Face!



An Eskimol



A Face!

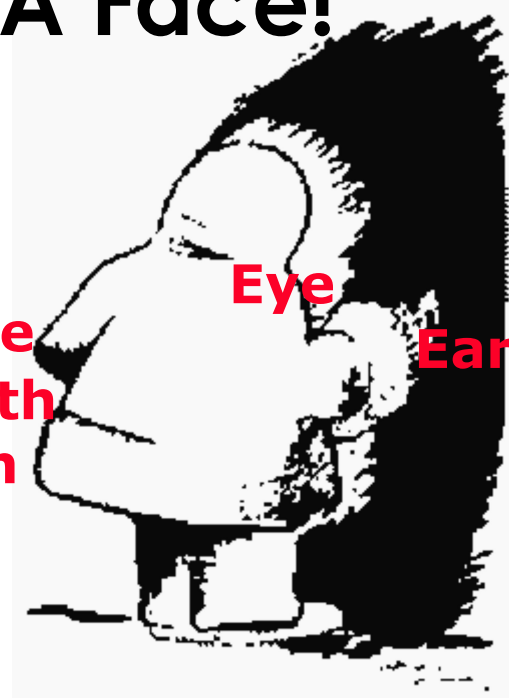
Cave

Nose
Mouth
Chin

Eye

Ear

Hair

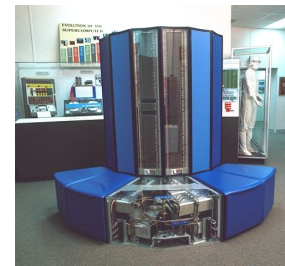


Class Identification (cont'd)

- **Why can we do this?**
 - Philosophy, science, experimental evidence
- **What are the limitations?**
 - Depending on the purpose of the system, different objects might be found
- **Crucial**
Identify the purpose of a system

Application vs Solution Domain

- **Application Domain** (Analysis):
 - The environment in which the system is operating
- **Solution Domain** (Design, Implementation):
 - The technologies used to build the system
- Both domains contain abstractions that we can use for the construction of the system model.



St	DEBN	Portfolio Size \$		St	DEBN	Portfolio Size \$	Common Data
St	DEBN	0.0000	(0.010, 0)	St	DEBN	0.0000	1.0000
5	DEBN	15,000	0.0000	5	DEBN	15,000	0.0000
6	DEBN	15,000	0.0000	6	DEBN	15,000	0.0000
7	DEBN	15,000	0.0000	7	DEBN	15,000	0.0000
8	DEBN	15,000	0.0000	8	DEBN	15,000	0.0000
9	DEBN	15,000	0.0000	9	DEBN	15,000	0.0000
10	DEBN	15,000	0.0000	10	DEBN	15,000	0.0000
11	DEBN	15,000	0.0000	11	DEBN	15,000	0.0000
12	DEBN	15,000	0.0000	12	DEBN	15,000	0.0000
13	DEBN	15,000	0.0000	13	DEBN	15,000	0.0000
14	DEBN	15,000	0.0000	14	DEBN	15,000	0.0000
15	DEBN	15,000	0.0000	15	DEBN	15,000	0.0000
16	DEBN	15,000	0.0000	16	DEBN	15,000	0.0000
17	DEBN	15,000	0.0000	17	DEBN	15,000	0.0000
18	DEBN	15,000	0.0000	18	DEBN	15,000	0.0000
19	DEBN	15,000	0.0000	19	DEBN	15,000	0.0000
20	DEBN	15,000	0.0000	20	DEBN	15,000	0.0000
21	DEBN	15,000	0.0000	21	DEBN	15,000	0.0000
22	DEBN	15,000	0.0000	22	DEBN	15,000	0.0000
23	DEBN	15,000	0.0000	23	DEBN	15,000	0.0000
24	DEBN	15,000	0.0000	24	DEBN	15,000	0.0000
25	DEBN	15,000	0.0000	25	DEBN	15,000	0.0000
26	DEBN	15,000	0.0000	26	DEBN	15,000	0.0000
27	DEBN	15,000	0.0000	27	DEBN	15,000	0.0000
28	DEBN	15,000	0.0000	28	DEBN	15,000	0.0000
29	DEBN	15,000	0.0000	29	DEBN	15,000	0.0000
30	DEBN	15,000	0.0000	30	DEBN	15,000	0.0000
31	DEBN	15,000	0.0000	31	DEBN	15,000	0.0000
32	DEBN	15,000	0.0000	32	DEBN	15,000	0.0000
33	DEBN	15,000	0.0000	33	DEBN	15,000	0.0000
34	DEBN	15,000	0.0000	34	DEBN	15,000	0.0000
35	DEBN	15,000	0.0000	35	DEBN	15,000	0.0000
36	DEBN	15,000	0.0000	36	DEBN	15,000	0.0000
37	DEBN	15,000	0.0000	37	DEBN	15,000	0.0000
38	DEBN	15,000	0.0000	38	DEBN	15,000	0.0000
39	DEBN	15,000	0.0000	39	DEBN	15,000	0.0000
40	DEBN	15,000	0.0000	40	DEBN	15,000	0.0000
41	DEBN	15,000	0.0000	41	DEBN	15,000	0.0000
42	DEBN	15,000	0.0000	42	DEBN	15,000	0.0000
43	DEBN	15,000	0.0000	43	DEBN	15,000	0.0000
44	DEBN	15,000	0.0000	44	DEBN	15,000	0.0000
45	DEBN	15,000	0.0000	45	DEBN	15,000	0.0000
46	DEBN	15,000	0.0000	46	DEBN	15,000	0.0000
47	DEBN	15,000	0.0000	47	DEBN	15,000	0.0000
48	DEBN	15,000	0.0000	48	DEBN	15,000	0.0000
49	DEBN	15,000	0.0000	49	DEBN	15,000	0.0000
50	DEBN	15,000	0.0000	50	DEBN	15,000	0.0000

Why do we need such models?

Reality: A stock exchange lists many companies. Each company is identified by a ticker symbol

Analysis results in analysis object model (UML Class Diagram):



Implementation results in source code (Java):

```
public class StockExchange {
    public m_Company = new Vector();
};

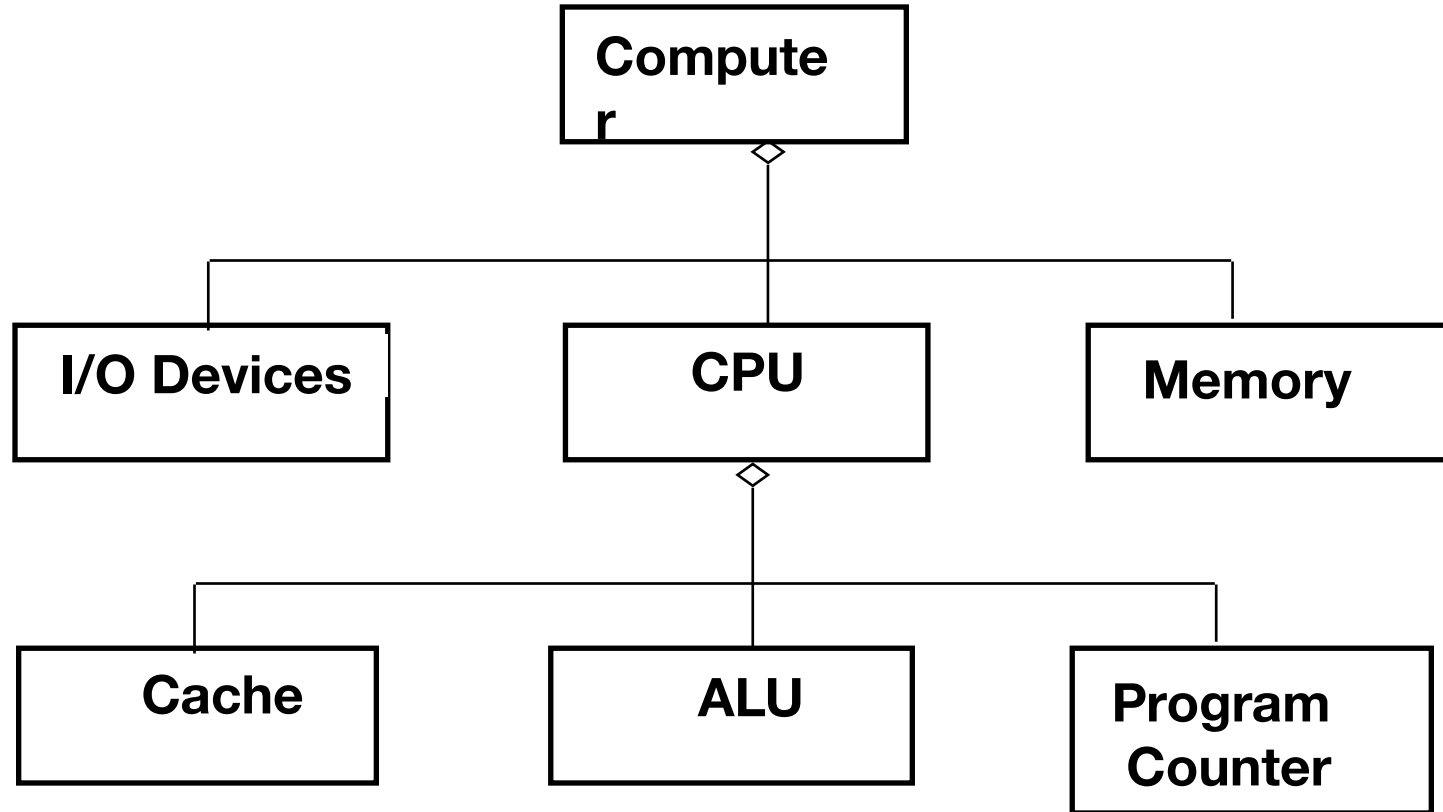
public class Company {
    public int m_tickerSymbol;
    public Vector m_StockExchange = new Vector();
};
```

- Three ways to deal with complexity
 - Abstraction and Modeling
 - Decomposition
 - **Hierarchy**

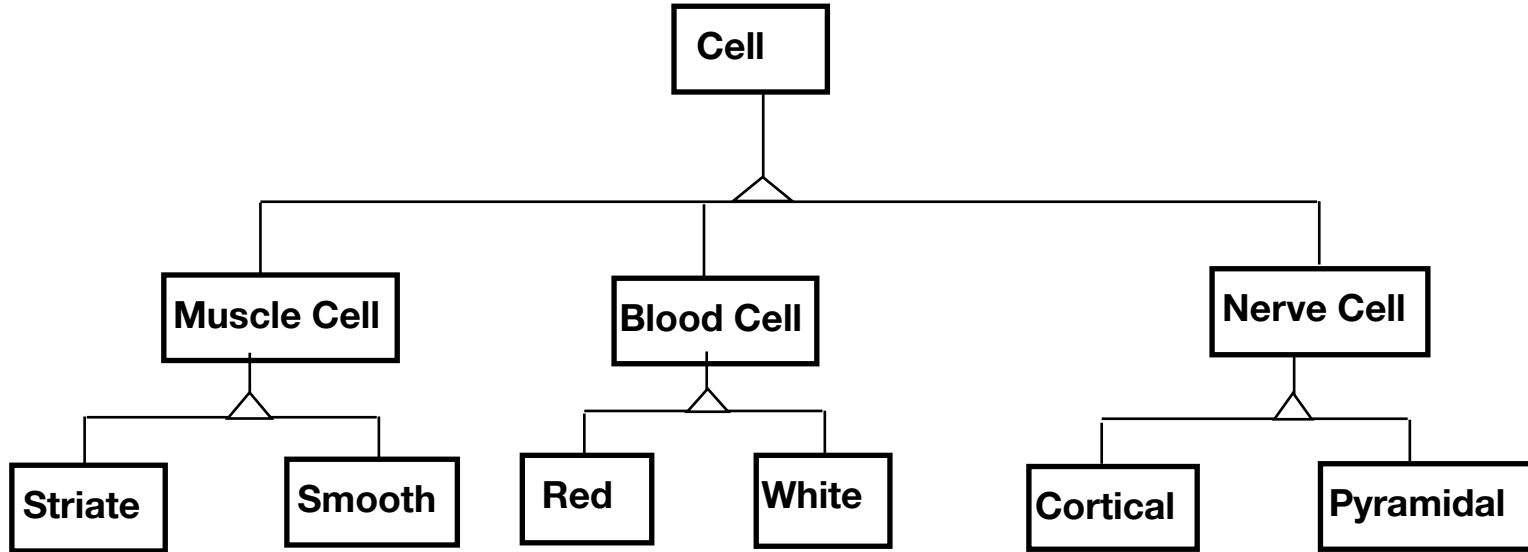
Hierarchy

- So far we got abstractions
 - This leads us to classes and objects
 - "Chunks"
- Another way to deal with complexity is to provide relationships between these chunks
- One of the most important relationships is hierarchy
- 2 special hierarchies
 - "Part-of" hierarchy
 - "Is-kind-of" hierarchy

Part-of Hierarchy (Aggregation)



Is-Kind-of Hierarchy (Taxonomy)



Where are we now?

- Three ways to deal with complexity:
 - Abstraction, Decomposition, Hierarchy
- Object-oriented decomposition is good
 - Unfortunately, depending on the purpose of the system, different objects can be found
- How can we do it right?
 - Start with a description of the functionality of a system
 - Then proceed to a description of its structure

What is UML?

- **UML** (Unified Modeling Language)
 - Nonproprietary standard for modeling software systems, OMG
 - Convergence of notations used in object-oriented methods
 - OMT (James Rumbaugh and colleagues)
 - Booch (Grady Booch)
 - OOSE (Ivar Jacobson)
- Commercial tools: Rational (IBM), Together (Borland), Visual Architect (business processes, BCD)
- Open Source tools: ArgoUML, StarUML, Umbrello
- Commercial and Opensource: PoseidonUML (Gentleware)

UML: First Pass

- You can model 80% of most problems by using about 20 % UML
- We teach you those 20%
- 80-20 rule: Pareto principle
 - http://www.ephorie.de/hindle_pareto-prinzip.htm

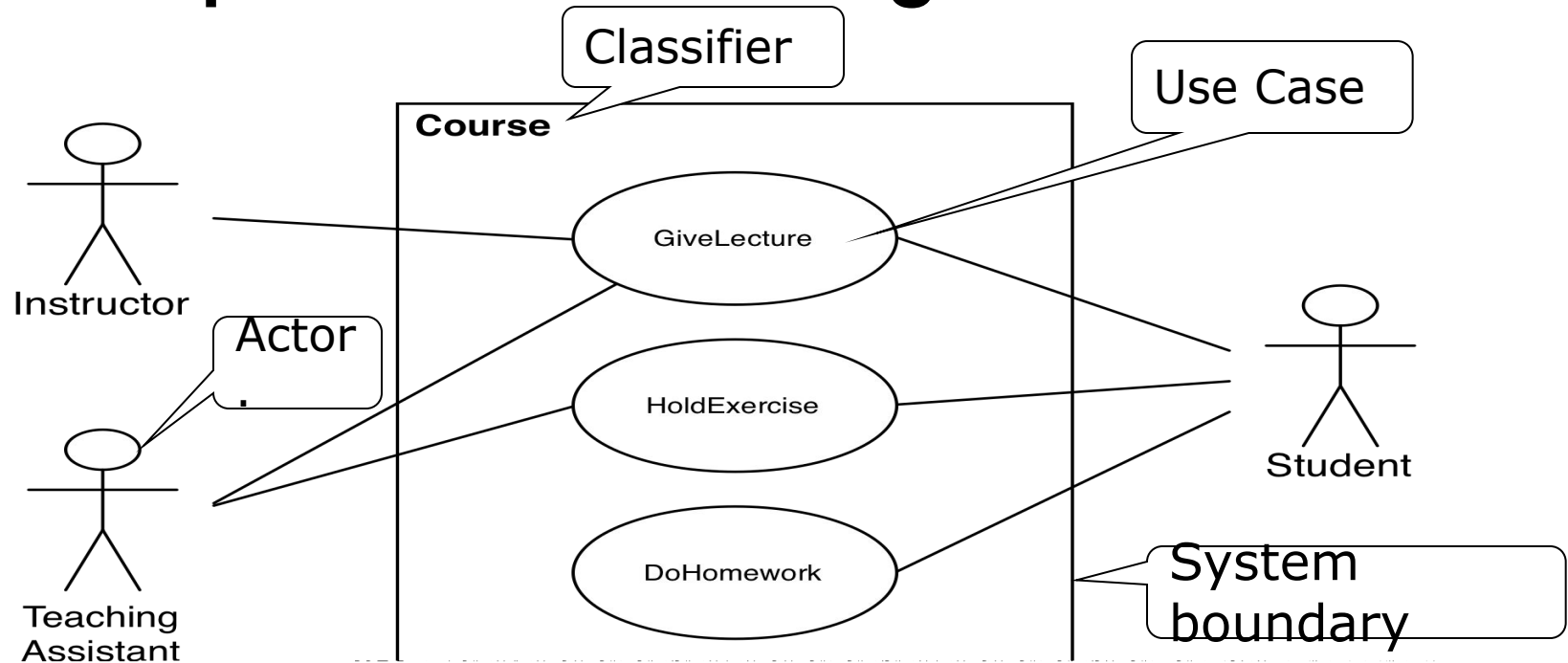
UML First Pass

- **Use case diagrams**
 - Describe the functional behavior of the system as seen by the user
- **Class diagrams**
 - Describe the static structure of the system: Objects, attributes, associations
- **Sequence diagrams**
 - Describe the dynamic behavior between objects of the system
- **Statechart diagrams**
 - Describe the dynamic behavior of an individual object
- **Activity diagrams**
 - Describe the dynamic behavior of a system, in particular the workflow.

UML Core Conventions

- All UML Diagrams denote graphs of nodes and edges
 - Nodes are entities and drawn as rectangles or ovals
 - Rectangles denote classes or instances
 - Ovals denote functions
- Names of Classes are not underlined
 - SimpleWatch
 - Firefighter
- Names of Instances are underlined
 - myWatch:SimpleWatch
 - Joe:Firefighter
- An edge between two nodes denotes a relationship between the corresponding entities

UML first pass: Use case diagrams

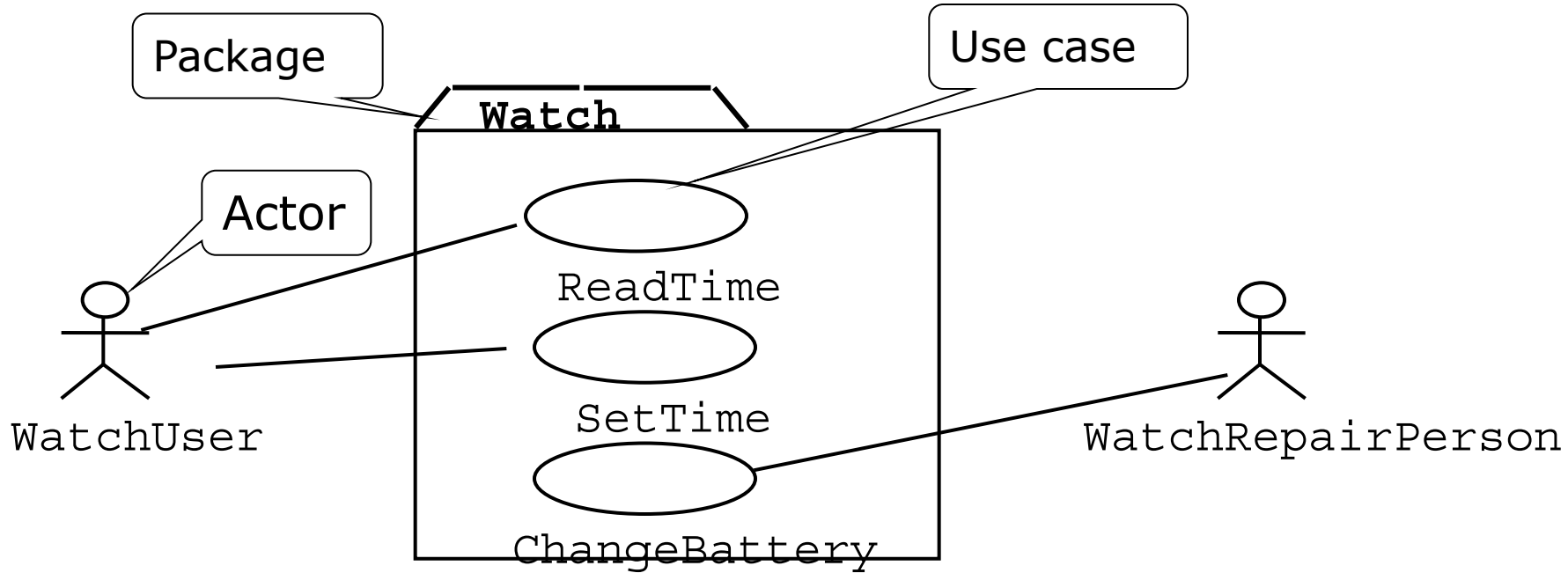


Use case diagrams represent the functionality of the system

Example: User Watch Application

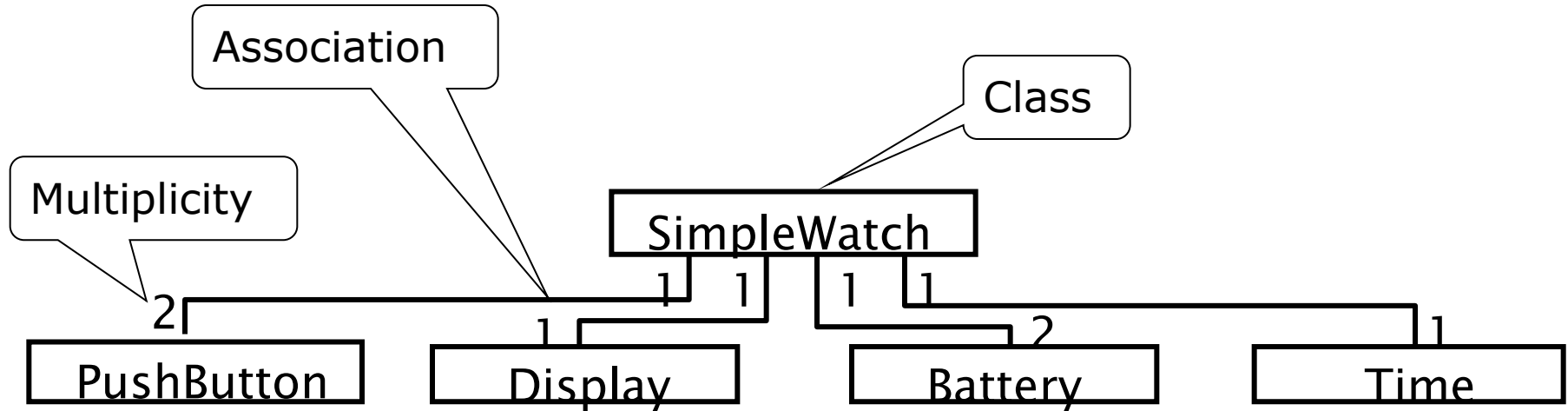
- Consider that you have a watch that can be used by a user for reading time, setting the time, and changing its battery
- We need to transform such description into some form that helps to develop such an application, **in a manner that reflects the customer's needs.**
- **How does this watch work?**

Historical Remark: UML 1 used packages



Use case diagrams represent the functionality of the system

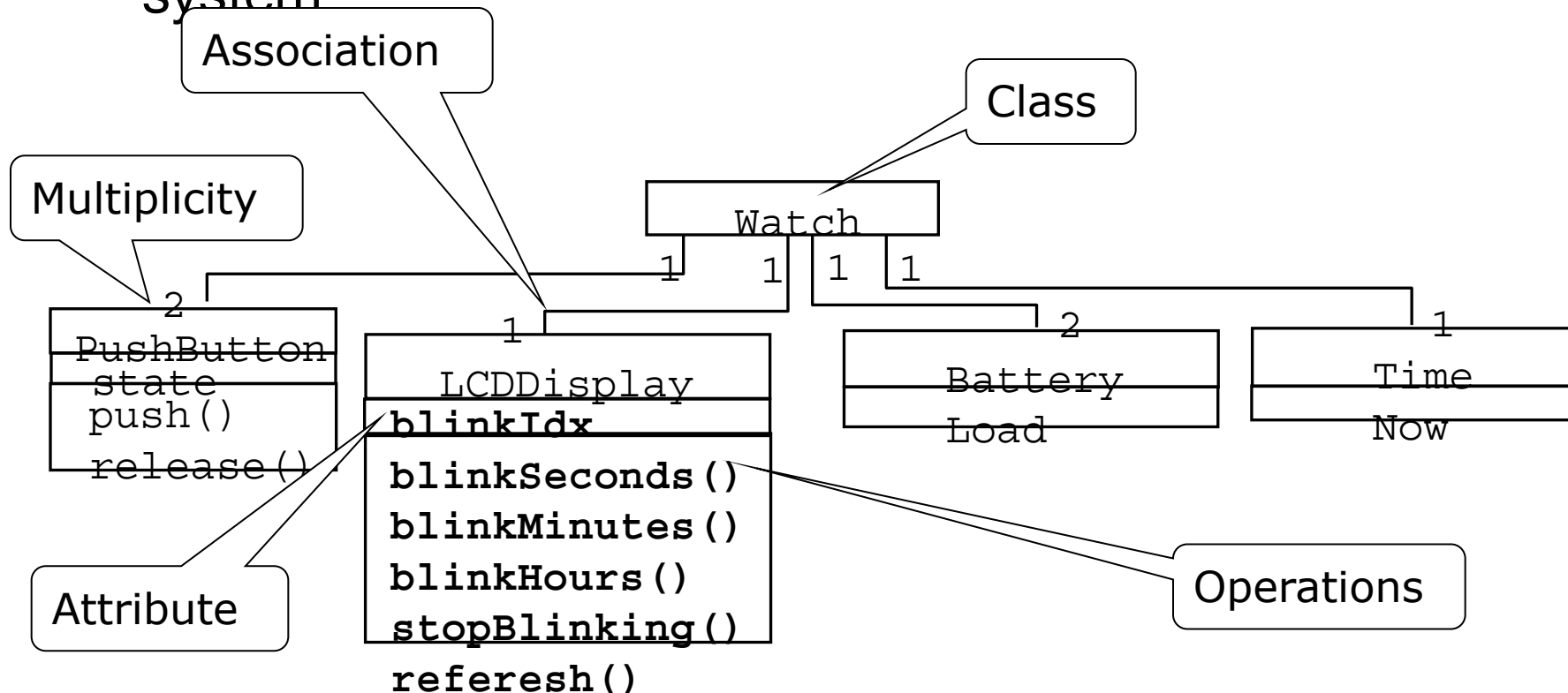
UML first pass: Class diagrams



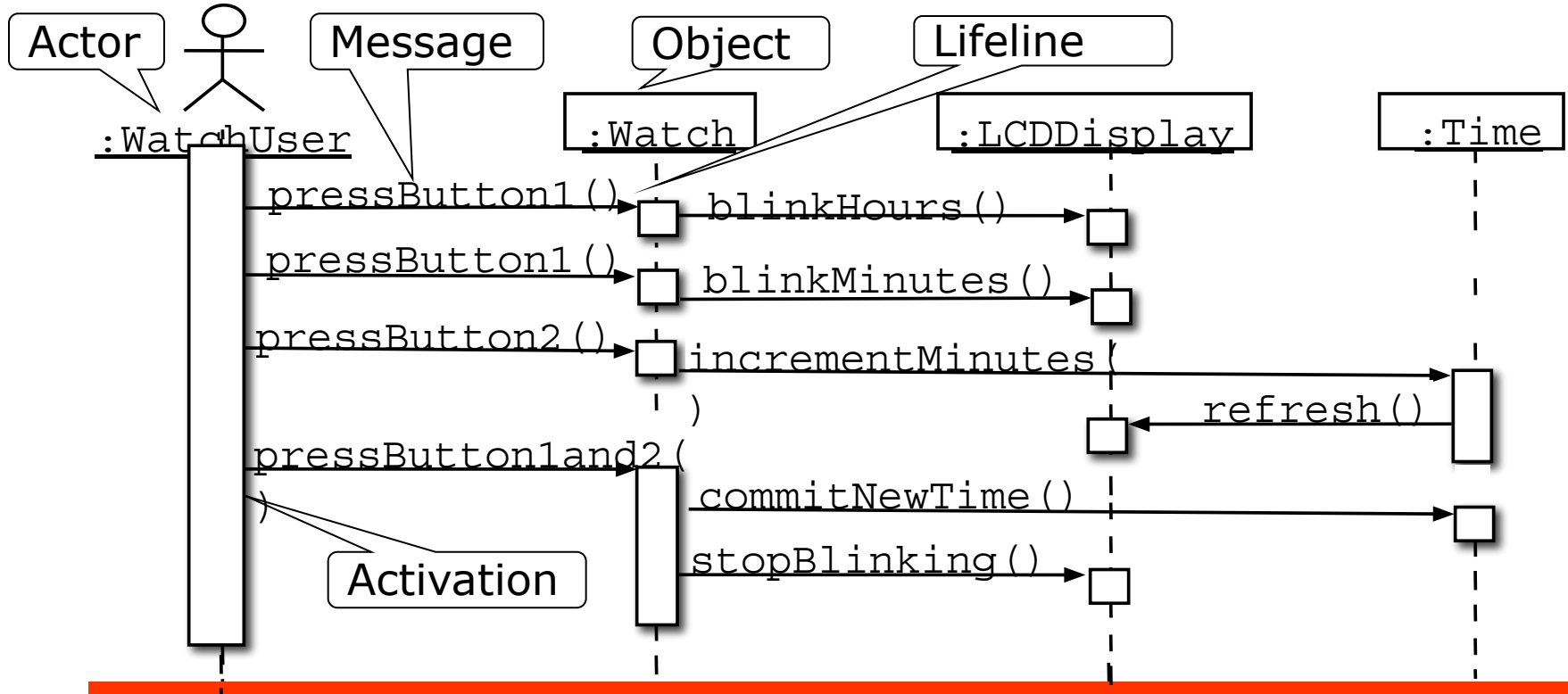
Class diagrams represent the structure of the system

UML first pass: Class diagrams

Class diagrams represent the structure of the system

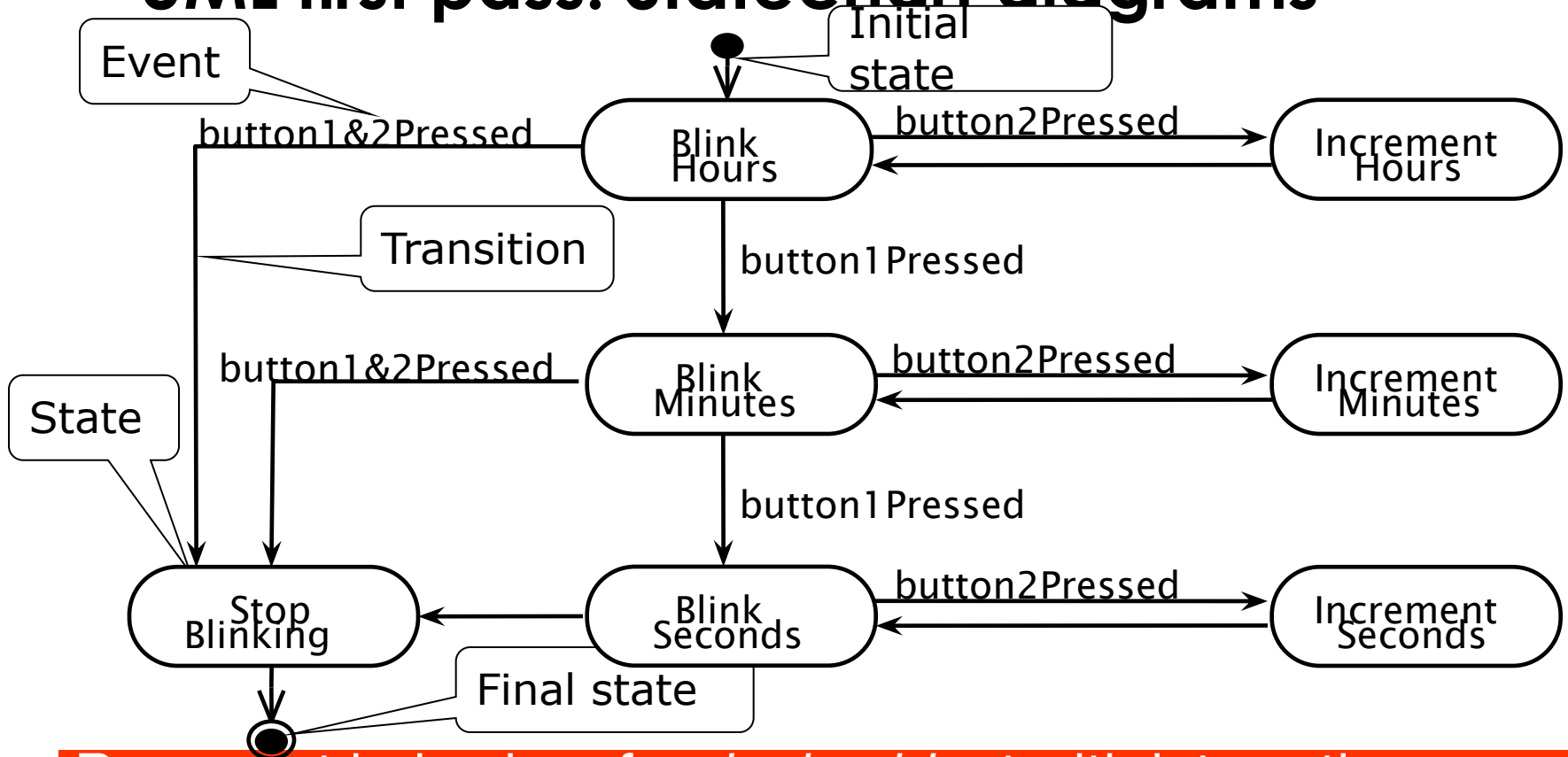


UML first pass: Sequence diagram



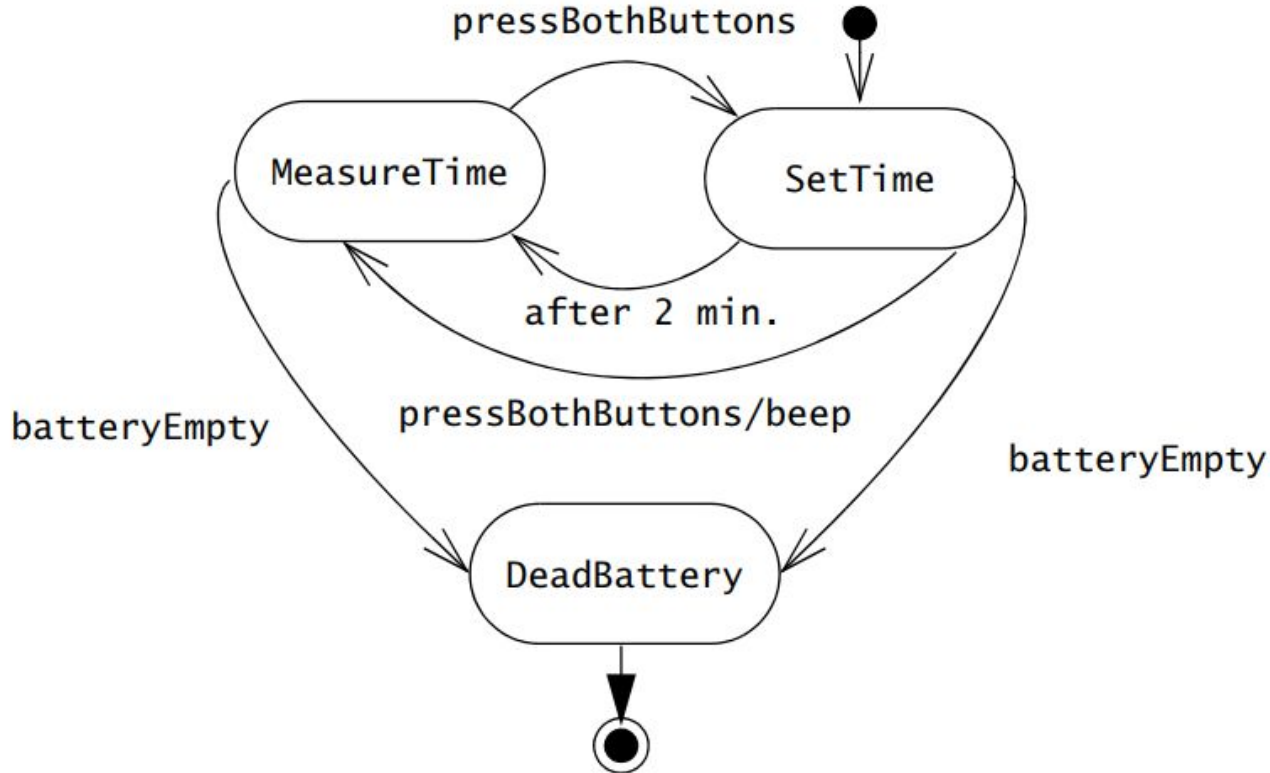
Sequence diagrams represent the behavior of a system as messages ("interactions") between different objects

UML first pass: Statechart diagrams



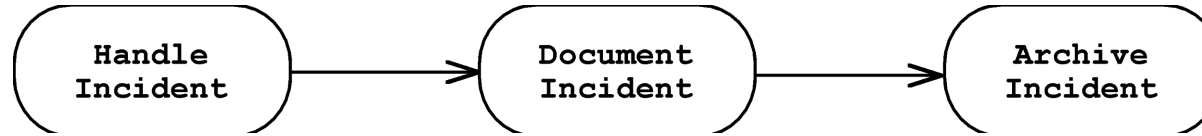
Represent behavior of *a single object* with interesting dynamic behavior.

2BWatch State Chart (Higher level abstraction)

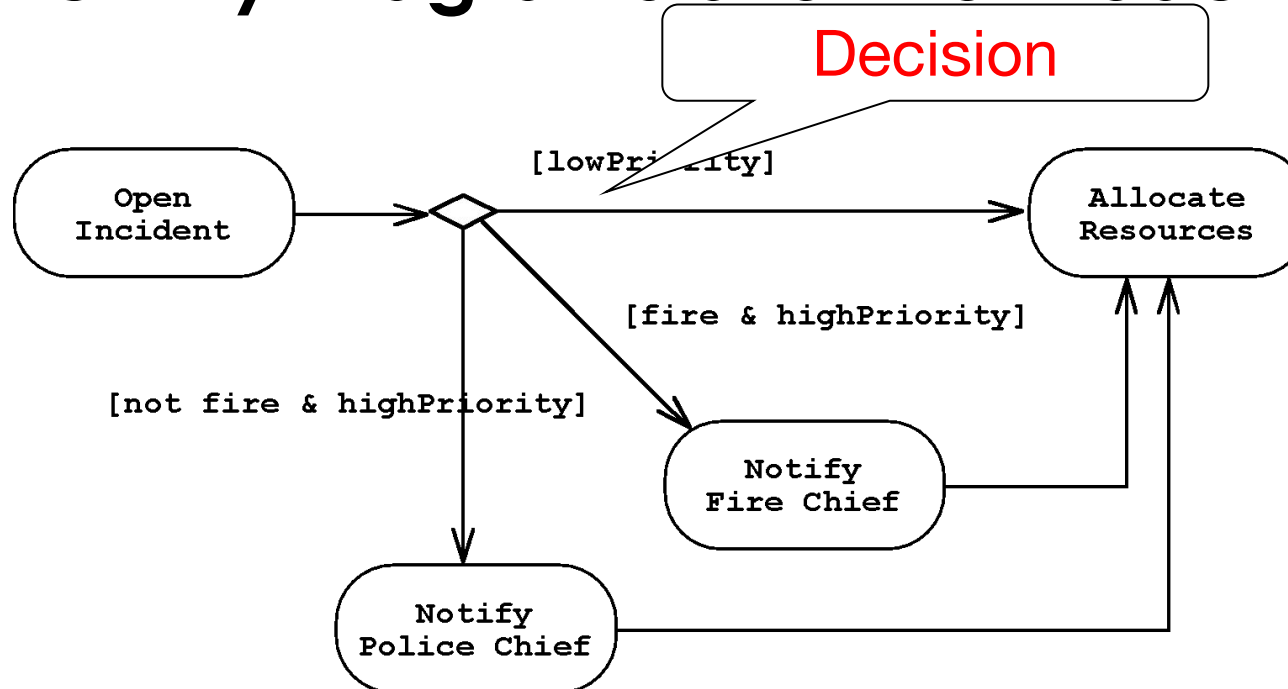


Activity Diagrams

- An activity diagram is a special case of a state chart diagram
- The states are activities (“functions”)
- An activity diagram is useful to depict the workflow in a system

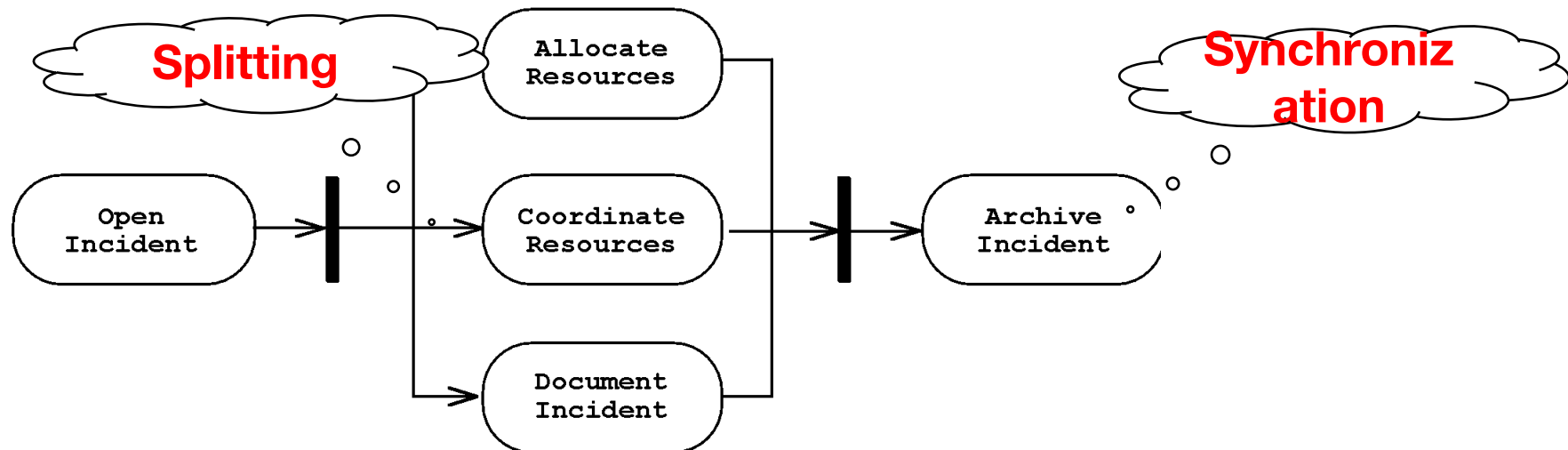


Activity Diagrams allow to model Decisions



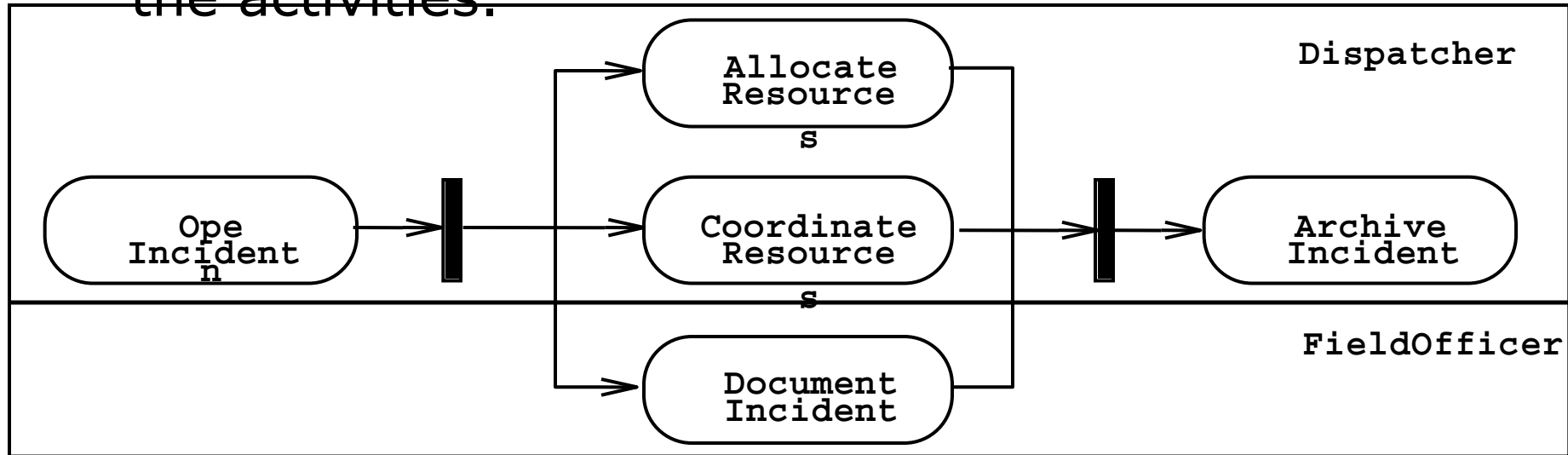
Activity Diagrams can model Concurrency

- Synchronization of multiple activities
- Splitting the flow of control into multiple threads



Activity Diagrams: Grouping of Activities

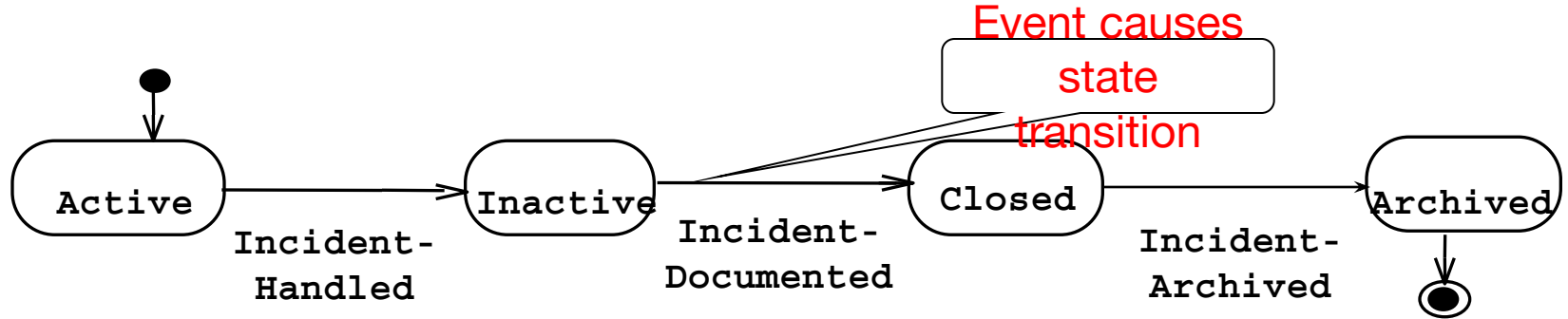
- Activities may be grouped into **swimlanes** to denote the object or subsystem that implements the activities.



Activity Diagram vs. Statechart Diagram

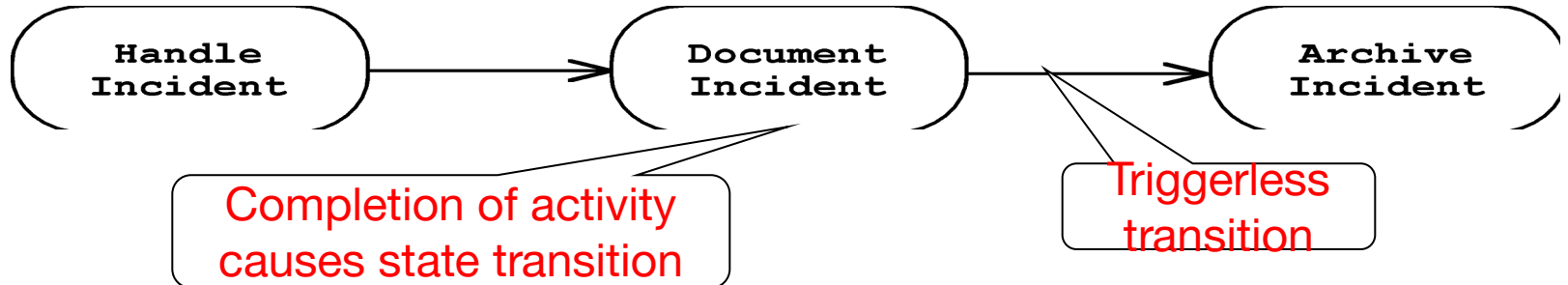
Statechart Diagram for Incident

Focus on the set of attributes of a single abstraction (object, system)



Activity Diagram for Incident

(Focus on dataflow in a system)



UML Summary

- UML provides a wide variety of notations for representing many aspects of software development
 - Powerful, but complex
- UML is a programming language
 - Can be misused to generate unreadable models
 - Can be misunderstood when using too many exotic features
- We concentrated on a few notations:
 - Functional model: Use case diagram
 - Object model: class diagram
 - Dynamic model: sequence diagrams, statechart and

Other UML Notations

UML provides many other notations, for example

- Deployment diagrams for modeling configurations
 - Useful for testing and for release management
- We introduce these and other notations as we go along in the lectures
 - OCL: A language for constraining UML models

Required Reading

- **Sections 1.1, 1.2, 1.3, 1.4, 2.1, and 2.2**
from the textbook: Bruegge, Bernd, and Allen H. Dutoit. "Object--Oriented Software Engineering. Using UML, Patterns, and Java." *Learning* 5.6 (2009): 7.